

GIT LECTURE NOTES

Based on the lecture by Sir Ameen Alam

Introduction to Git

Git is a distributed version control system (VCS) that helps developers manage and track changes in their source code. It is widely used for collaboration, allowing multiple developers to work on the same project without conflicts. Git operates in three main stages:

- **Stage** – Preparing changes to be committed.
- **Commit** – Saving changes to the local repository.
- **Remote** – Uploading committed changes to a remote repository (e.g., GitHub, GitLab).

Understanding Git Workflow

1. Changes & Staging

When a developer modifies a file, it is considered a change. These changes must be staged before committing. Staging allows developers to select specific files or changes to be included in the next commit.

```
git add .          # Stages all modified files
git add file      # Stages a specific file
```

2. Commit

Once changes are staged, they are committed with a meaningful message. This creates a snapshot of the changes in the local repository.

```
git commit -m "Added new feature"
```

3. Pushing to Remote Repository

The committed changes are then pushed to a remote repository (e.g., GitHub, GitLab) for collaboration and backup.

```
git push origin
```

Git Architecture

Git divides its architecture into two main areas:

- **Local Machine** – Where the developer writes, stages, and commits code.
- **Remote Repository** – Where the committed code is stored and shared with others.

Popular remote platforms include:

- GitHub
- GitLab
- Bitbucket
- Azure DevOps

Branching in Git

Branching allows developers to work on different features or fixes without affecting the main codebase.

Creating and Switching Branches

```
git checkout -b feature-branch    # Create a new branch
git checkout main                 # Switch to an existing branch
git branch -a                    # View all branches (local and remote)
```

Merging Branches

Once a feature is complete, the branch can be merged into the main branch.

```
git merge feature-branch
```

Cloning and Pulling a Repository

```
git clone                # Clone a repository
git pull origin          # Pull the latest changes from the remote repository
```

Tracking Changes & Logs

```
git status          # Check the status of modified files
git log             # View the commit history
git branch          # View the current branch
```

Reverting and Resetting Changes

1. **Undoing the Last Commit (Soft Reset):** Keeps the changes but removes the last commit.

```
git reset --soft HEAD~1
```

2. **Undoing the Last Commit (Hard Reset):** Deletes the last commit and all changes permanently.

```
git reset --hard HEAD~1
```

3. **Removing a Branch:**

```
git branch -D feature-branch    # Delete a branch locally
git push origin --delete feature-branch  # Delete a branch remotely
```

4. **Removing Files from Staging:**

```
git reset          # Unstage a specific file
git reset          # Unstage all files
git reset --hard    # Remove all uncommitted changes
```

Force Pushing and Overwriting Remote

```
git push origin --force  # Force push and overwrite remote
```

Git Configuration

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

Summary of Git Commands

Command	Description
git add .	Stage all modified files
git add	Stage a specific file
git commit -m "message"	Commit staged changes with a message
git push origin	Push committed changes to remote
git clone	Clone a repository
git pull origin	Pull latest changes from remote
git status	Check status of files
git log	View commit history
git branch -a	View all branches
git checkout -b new-branch	Create and switch to a new branch
git merge feature-branch	Merge a branch into the main branch
git reset --soft HEAD~1	Undo last commit but keep changes
git reset --hard HEAD~1	Undo last commit and delete changes
git reset	Unstage a specific file
git reset	Unstage all files
git reset --hard	Remove all uncommitted changes
git branch -D feature-branch	Delete a branch locally
git push origin --delete feature-branch	Delete a branch remotely

git push origin --force	Force push and overwrite remote
-------------------------	---------------------------------

Conclusion

Git is a powerful tool for version control and collaboration. By understanding its core concepts—staging, committing, pushing, branching, and resetting—developers can efficiently manage their code and work seamlessly in teams. The branching strategy ensures that multiple developers can work on different features without conflicts, and the ability to revert changes provides a safety net for mistakes.